

Trust boundaries, traffic paths and listening ports

Three views of the same deployment. The first follows traffic inward from the public internet to the ports the gateway publishes; the second follows it between components once inside; the third shows where each component system can point — the NEC-hosted instance or a partner-hosted one at another IP. Every link is labelled with the port it actually uses.

● Public internet

● Internal network / gateway

● Host loopback only

● Container network only

● Partner-hosted, another IP

FIGURE 1 · NORTH-SOUTH

Ingress path — internet to published service ports

Three ports face the internet. The gateway terminates TLS and forwards to the loopback address of each service, so no application port is ever exposed at the firewall.

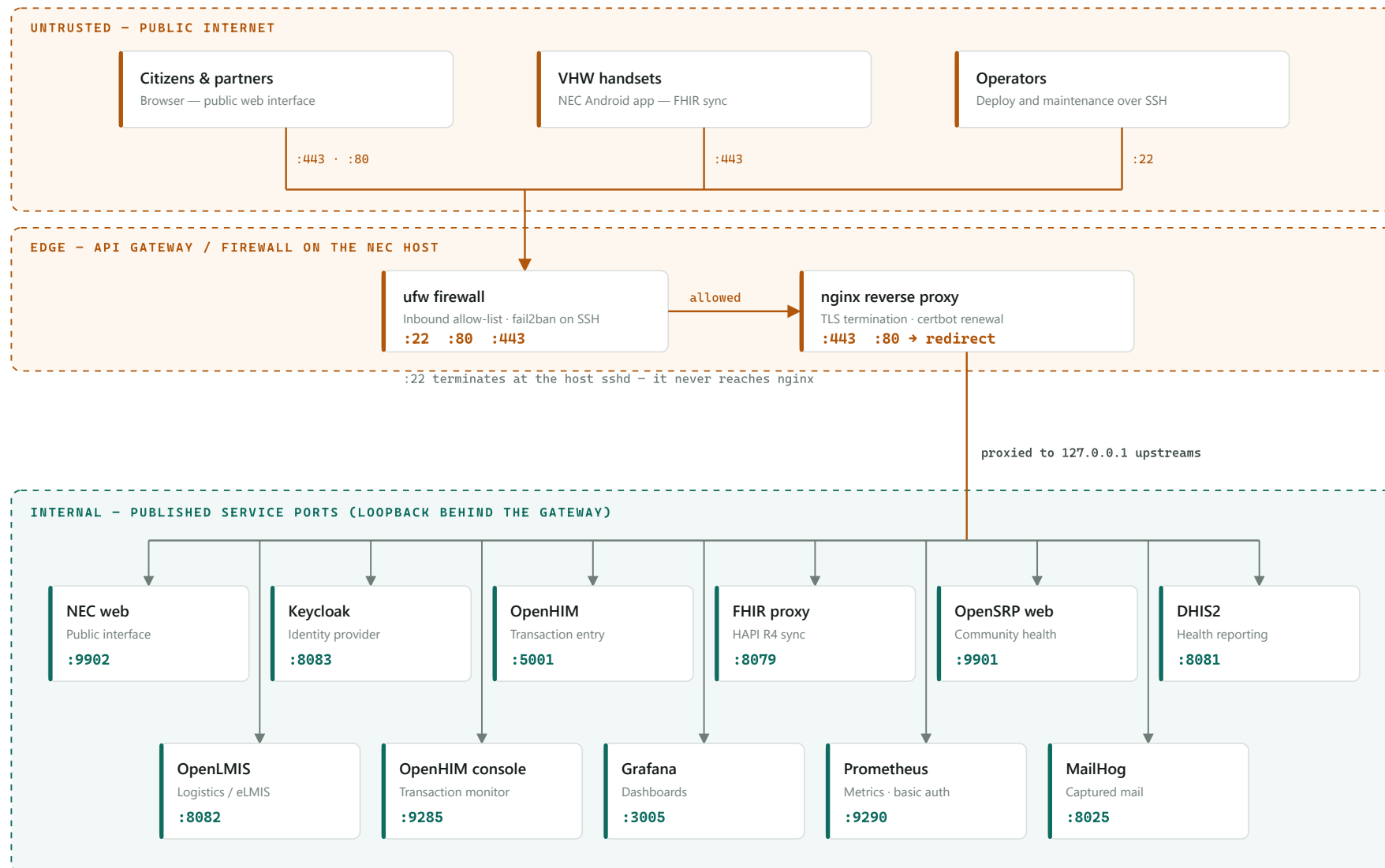


FIGURE 2 · EAST-WEST

Internal path — orchestration, component systems and data tier

Once inside, traffic is container-to-container on the Docker networks. Ports shown on each link are the ones the connection actually dials; where a service is also published on the host, the host port is noted beneath its name.

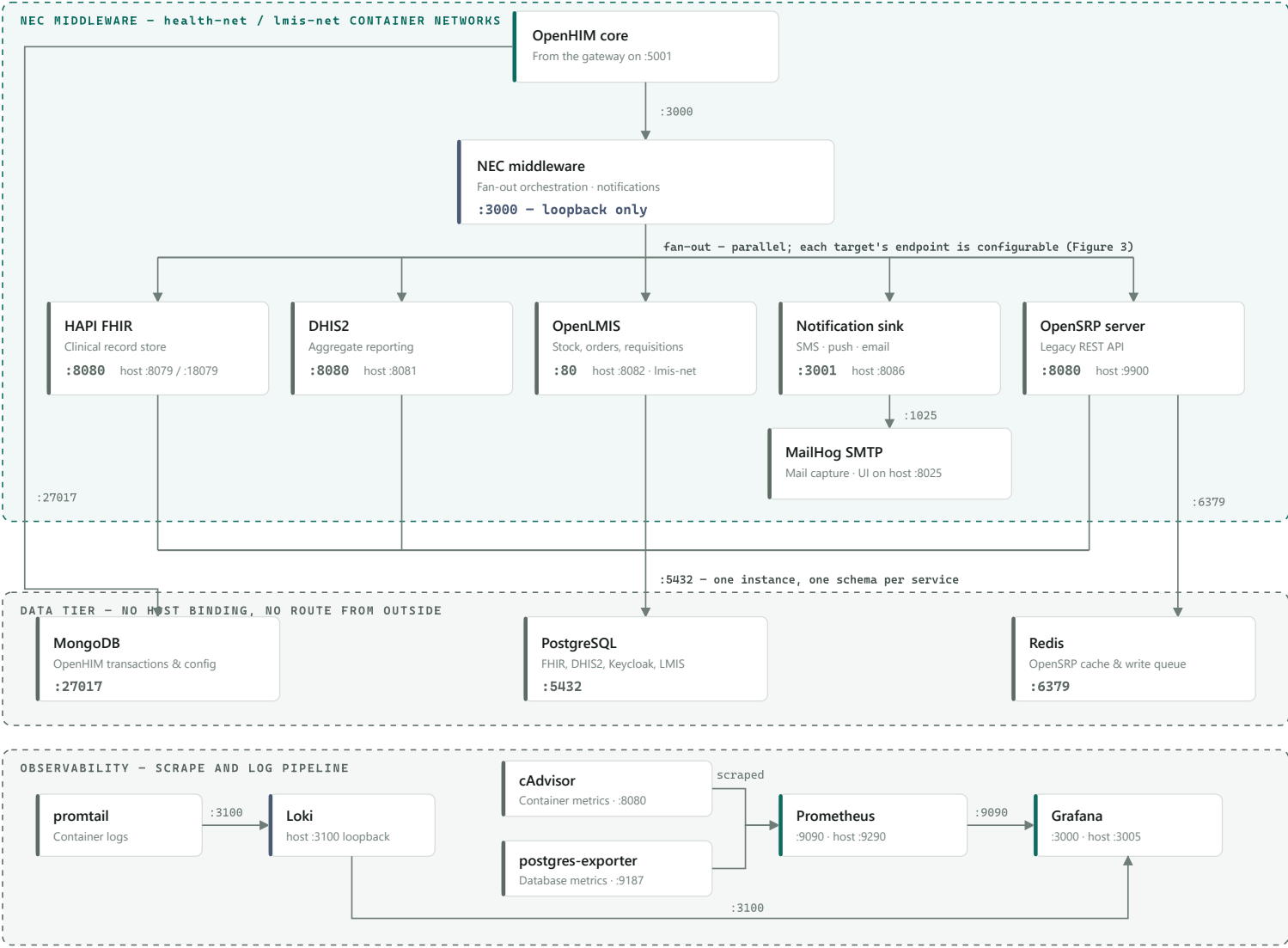
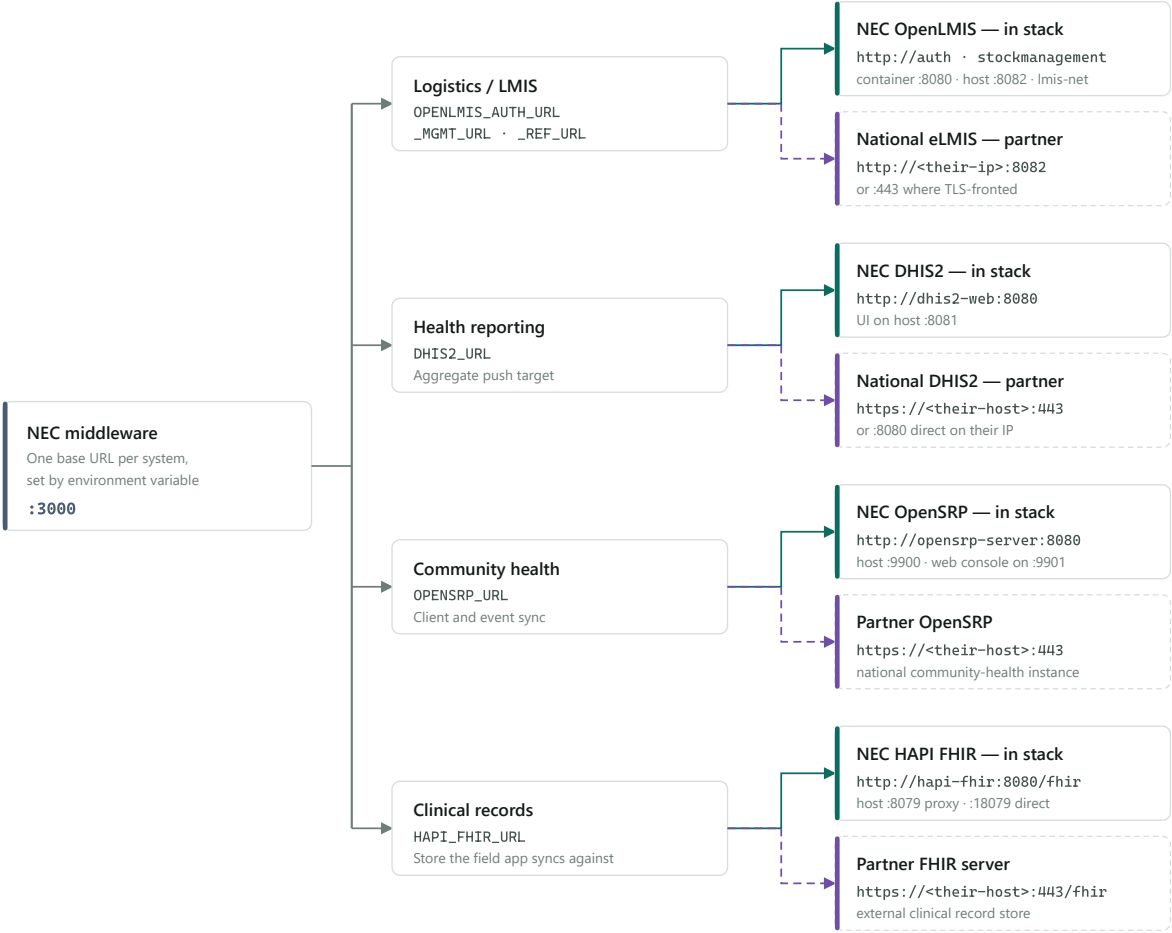


FIGURE 3 · UPSTREAM SELECTION

Ours or theirs — each component system points at one endpoint

The middleware reaches every component system through a single configured base URL, so each one can point either at the NEC-hosted instance in this stack or at a partner-hosted instance on another IP. The choice is made per system, not for the stack as a whole — the LMIS can be theirs while DHIS2 and OpenSRP stay ours.

SOLID — NEC-HOSTED, INSIDE THIS STACK
DASHED — PARTNER-HOSTED, ANOTHER IP



Partner endpoints are outbound calls from the middleware — they need egress to that IP and port, not an inbound firewall rule.

READING THE DIAGRAMS

- Dashed outlines are trust boundaries, not subnets — everything below the edge runs on one host.
- A port on a node is what that service listens on; a port on a link is what the caller dials.
- Ports written as `host :8081` are the host publication of a container port.

DEPLOYMENT VARIANTS

- Domain server: only 22, 80 and 443 open; Figure 1's service ports stay on loopback.
- IP-only server with no domain: those eleven service ports are opened directly with `ufw allow` and served over plain HTTP.
- Sandbox on a laptop: same bindings, no firewall in front.
- Mixed hosting: any component system can be repointed at another IP without touching the rest of the stack.

POINTING A SYSTEM ELSEWHERE

- Set the system's base URL on the middleware — `OPENLMIS_AUTH_URL` , `OPENLMIS_MGMT_URL` , `OPENLMIS_REF_URL` , `DHIS2_URL` , `OPENSERP_URL` or `HAPI_FHIR_URL` .
- The repo default points all six at the in-stack instances; using a partner system is a per-deployment override, not a code change.
- The in-stack containers can then be left out of the compose file, freeing their host ports.
- Credentials follow the endpoint — a partner system needs its own client and token settings.

SOURCE OF TRUTH

- `docker-compose.yml` — bindings and container ports
- `deploy/nginx/` — gateway routing
- `deploy/setup-server.sh` — firewall allow-list
- `config/prometheus/` — scrape targets